

NET Versions, Namespaces, Projects & Solutions

First: .NET Versions

.NET has gone through several major stages.

.At first, there was **.NET Framework**, which was mainly designed for Windows

Then **.NET Core** was introduced with a major goal: making .NET **cross-platform**, so applications could run on Windows, Linux, and macOS

.Starting with **.NET 5**, Microsoft unified the platform under the name **.NET**

:After that, we got versions like

.NET 6 → .NET 7 → .NET 8 → .NET 9 → .NET 10.

So, **.NET Core isn't the name of modern .NET versions**. It was the name of the platform before the unification that started with .NET 5

Second: Namespaces

A **Namespace** is basically a way to organize Classes and Types in your application and avoid naming conflicts

:For example, you could have

`Company.Models.User`

:and

`Company.Services.User`

.Both Classes can be called `User`, but they belong to different Namespaces

.To use a Type from another Namespace, you can use the **using** directive

.This keeps your code cleaner instead of having to write the full Namespace every time

Third: Projects & Solutions

A **Project** is a unit that contains the code, files, dependencies, and configuration needed to build a specific part of your application

For example, you might have

- Web API Project
- Business Logic Project
- Data Access Project
- Tests Project

A **Solution**, on the other hand, is a container that can hold multiple related Projects

So one Solution can contain several Projects, where each Project is responsible for a specific part of the application

.NET Version → Defines the platform version, APIs, and available features.

.Namespace → Organizes Types and helps avoid naming conflicts

.Project → Represents a buildable part of the application

.Solution → Groups related Projects together